

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA1001	Course Title:	Software Engineering
CO Assessed:	CO1	Assessment:	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	Total Marks:	25
CO1 Statement:	Apply advanced methodologies and agile project management techniques to manage software projects effectively		
Student Name:	M. Shanmukha Aditya	Reg. No.:	192365055
Date:		Duration:	60 minutes

Code of Conduct

I M. Shanmukha Aditya(192365055) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M. Aditya

Annexure A – Case Study Analysis

CASE STUDY REPORT

Smart Automation & Cloud Architecture

Designing a Cloud-Native Smart Automation Platform for a Multi-Site Enterprise

1. Understanding the Problem Statement

1.1 The Problem in My Own Words

Modern organisations such as manufacturing plants, logistics hubs, hospitals, campuses and smart buildings run hundreds of machines, sensors and business processes every day. In most of them these processes are still operated manually or through isolated, on-premise control systems. Operators walk the floor to take readings, supervisors approve routine tasks on paper or by phone, maintenance happens only after something breaks, and data from each site stays locked in its own local server or spreadsheet.

The problem addressed in this case study is: **how can an organisation use smart automation (IoT sensing, rule-based and AI-driven workflows) together with a well-designed cloud architecture so that its operations become automatic, connected, observable, secure and able to grow without a redesign?** In short, the organisation needs to move from "people reacting to problems" to "software detecting, deciding and acting", while keeping the system cheap to scale, easy to maintain and safe against failures and cyber-attacks.

For this study, the problem is grounded in a realistic scenario: **a mid-sized manufacturing enterprise with five plants in different cities** that wants to automate monitoring, alerts, maintenance scheduling, energy control and reporting across all plants from one cloud-hosted platform, which we call the **SmartCloud Automation Platform (SCAP)**.

1.2 Objectives of the Proposed System

- **Automate routine operations** such as data collection, threshold checks, alerts, approvals and work-order creation, removing manual and repetitive effort.

- **Centralise data** from all plants, machines and enterprise systems (ERP, MES, CMMS) into one reliable cloud data platform that gives a single version of the truth.
- **Enable real-time monitoring and control** through dashboards, live alerts and remote commands to devices.
- **Reduce downtime through predictive maintenance** by using machine-learning models on sensor data to predict failures before they occur.
- **Optimise energy and resource use** through automated scheduling and anomaly detection.
- **Provide elastic scalability and high availability** so that new plants, devices and users can be added without redesigning the system and without service interruption.
- **Ensure security and compliance** through authentication, role-based access, encryption and audit trails.
- **Reduce total cost of ownership** by using pay-as-you-go cloud services instead of heavy upfront hardware investment.

1.3 Scope of the Problem

In scope

- Collection of data from sensors, PLCs and meters through edge gateways and secure protocols (MQTT, OPC-UA, HTTPS).
- Cloud-based ingestion, storage, processing and analytics of structured and time-series data.
- A rule and workflow engine for automatic alerts, approvals, device commands and maintenance work orders.
- Predictive-maintenance and energy-analytics modules using machine learning.
- Web and mobile dashboards, report generation and role-based user management.
- Integration with existing ERP / MES / CMMS systems through APIs.
- Security, monitoring, backup, disaster recovery and a CI/CD pipeline for the platform itself.

Out of scope

- Manufacture or replacement of physical machines, sensors and PLC hardware (existing devices are reused through adapters).
- Redesign of the plant's physical production process or layout.
- Fully autonomous control of safety-critical equipment without human override (safety interlocks remain on the local controller).

2. Key Stakeholders and Challenges

2.1 Stakeholders, Roles and Responsibilities

Stakeholder	Role and Responsibilities	Main Challenges Faced
Senior Management / Business Owners	Define business goals and budget; approve the project; track ROI, productivity and compliance.	Lack of a single, trustworthy view of operations; unclear cost and return on investment; fear of vendor lock-in.
Plant / Operations Managers	Run day-to-day production; act on alerts; approve schedules; ensure targets and safety are met.	Late information, manual reports, unplanned downtime, difficulty comparing plants.
Machine Operators / Technicians	Operate machines, record readings, respond to alarms and perform repairs.	Manual data entry errors, alarm overload, minimal training on new digital tools.
Maintenance Engineers	Plan preventive maintenance, diagnose faults, manage spare parts and work orders.	Reactive "breakdown" maintenance, no failure prediction, poor maintenance history.

Stakeholder	Role and Responsibilities	Main Challenges Faced
IT / Cloud / DevOps Team	Design, deploy, monitor and secure the cloud platform; manage releases, backups and cost.	Legacy-system integration, cloud cost control, uptime targets, constant patching.
Software Development & QA Team	Gather requirements, design, build, test and maintain SCAP modules.	Changing requirements, integration testing with real devices, technical debt.
Data Scientists / Analysts	Build and monitor ML models, define KPIs, produce insights.	Poor data quality, scattered data sources, model drift.
Security & Compliance Officers	Define security policy; manage access control, audits and regulatory compliance.	Growing attack surface from IoT devices, data privacy rules, audit evidence.
Customers and Suppliers	Receive products and delivery information; supply materials and spare parts.	Delayed order status, quality variations, lack of real-time visibility.
Cloud and Hardware Vendors / Regulators	Provide cloud services, devices and support; define legal and safety standards.	Service-level agreements, interoperability, changing regulations.

2.2 Major Challenges and Issues

- **Data silos:** every plant stores data in its own system and format, so management cannot compare or combine information.
- **Heavy manual effort and human error:** readings, approvals and reports depend on people, which is slow and error-prone.
- **Unplanned downtime:** machines fail without warning because maintenance is calendar-based or reactive.
- **Poor scalability of on-premise servers:** adding a plant or a thousand new sensors needs new hardware, long procurement and downtime.
- **Legacy and heterogeneous devices:** machines from different vendors use different protocols (Modbus, OPC-UA, proprietary), making integration hard.
- **Security risk:** unmanaged IoT devices, shared passwords and flat networks are easy targets for cyber-attacks.
- **Reliability and connectivity:** plants in remote areas face unstable internet, so a purely cloud-dependent design could stop working when the link drops.
- **Cost uncertainty:** cloud bills can grow unexpectedly without governance and monitoring.
- **Change management:** staff may resist new tools unless the interface is simple and training is provided.

3. Analysis of the Current Approach

3.1 Description of the Existing System / Process

In the existing setup each plant operates as an independent unit. The typical flow is as follows:

1. Sensors and meters feed a local SCADA/PLC system, which shows readings only on a control-room screen.
2. Operators note readings on log sheets or enter them into spreadsheets at the end of every shift.
3. Supervisors consolidate the sheets and e-mail weekly or monthly reports to head office.
4. Maintenance follows a fixed calendar or begins only after a breakdown; work orders are raised by phone or on paper.

5. The on-premise server in each plant hosts a local database and ERP client; backup is manual and stored in the same building.
6. Approvals (purchase of spares, shift changes, quality release) move through e-mail and signatures.

3.2 Strengths of the Current Approach

- **Familiar and proven:** staff know the tools, so there is no learning curve.
- **Low latency for local control:** PLC/SCADA control loops run locally and respond instantly.
- **Works without internet:** local operations continue even if the WAN link is down.
- **Data stays on site:** gives a sense of control over data location and privacy.
- **No recurring cloud charges:** costs are mostly one-time capital expenditure that is already paid.

3.3 Limitations of the Current Approach

- **No real-time, enterprise-wide visibility:** data reaches management days or weeks late.
- **Reactive maintenance:** failures are noticed after they happen, causing downtime and rush repair costs.
- **Limited scalability:** capacity is fixed by the on-premise hardware; expansion is slow and expensive.
- **Single point of failure:** one local server crash or one fire/flood can destroy data and stop reporting.
- **Weak security:** outdated operating systems, no central patching, little monitoring, and no audit trail.
- **High maintenance burden:** each site needs its own IT support and software upgrades.
- **No intelligence:** data is stored but never analysed, so patterns, anomalies and energy waste remain hidden.
- **Poor integration:** ERP, SCADA, CMMS and spreadsheets do not exchange data automatically.

3.4 Gap Analysis

Area	Current State (As-Is)	Required State (To-Be)	Gap to Address
Data collection	Manual entry, shift-wise	Automatic, continuous, secure streaming	Edge gateways, IoT protocols
Data storage	Local servers, siloed	Centralised cloud data platform	Cloud DB + data lake
Maintenance	Reactive / calendar-based	Predictive and condition-based	ML models, alerts
Workflows	Phone, paper, e-mail	Automated rule-driven workflows	Workflow engine
Scalability	Hardware-bound	Elastic, on-demand	Containers, auto-scaling
Availability	Single site, manual backup	99.9%+ with DR	Multi-zone deployment
Security	Weak, unmanaged	Zero-trust, encrypted, audited	IAM, TLS, logging
Reporting	Weekly / monthly spreadsheets	Live dashboards and analytics	BI and API layer

4. Proposed Solution Using Software Engineering Principles

4.1 Solution Overview: SmartCloud Automation Platform (SCAP)

SCAP is a **cloud-native, event-driven, layered platform** that connects devices at every plant to a central cloud, applies automation rules and machine-learning intelligence, and presents results through secure web and mobile applications. It follows a hybrid **edge + cloud** approach: time-critical control stays on the local

controller and edge gateway, while storage, analytics, orchestration and enterprise integration run in the cloud. This preserves the strengths of the current approach (low latency, offline operation) while removing its limitations.

The architecture is organised in five layers with cross-cutting security, monitoring and DevOps concerns, as shown in Figure 1.

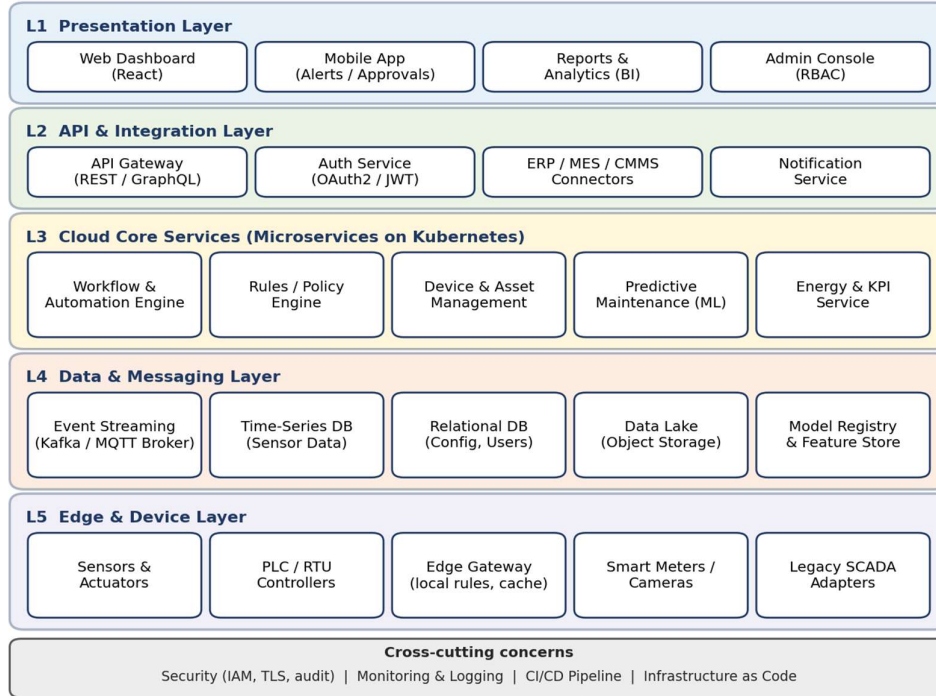


Figure 1: Layered Architecture of the Smart Automation & Cloud Platform (SCAP)

How the system works (data and control flow)

1. Sensors, PLCs and meters send readings to the **edge gateway**, which filters, buffers and forwards them securely (MQTT over TLS) to the cloud; it keeps local rules running if the internet fails.
2. The **event-streaming layer** receives data, validates it and stores it in the time-series database and data lake.
3. The **rules and workflow engine** evaluates each event (for example, "bearing temperature above limit for 5 minutes") and triggers actions: alert, work order, approval request or a command back to the device.
4. The **ML service** continuously scores equipment health and raises a predictive-maintenance ticket before failure.
5. Users see live status, trends and KPIs in the **dashboard**; ERP/MES/CMMS are updated automatically through the **integration layer**.

4.2 Major Functional Modules

Module	Description and Key Functions
1. Device & Edge Management	Registers devices, manages gateway configuration, firmware updates and certificates; protocol adapters for Modbus, OPC-UA, MQTT; local buffering and offline rules.
2. Data Ingestion & Streaming	Secure, scalable intake of sensor and system events; validation, enrichment and routing to storage and processing.
3. Data Storage & Management	Time-series DB for sensor data, relational DB for users/assets/configuration, object storage data lake for history and ML training; retention and backup policies.

Module	Description and Key Functions
4. Rules & Workflow Automation Engine	Low-code rule builder, thresholds, schedules, approval chains, automatic work-order creation, notifications and device commands.
5. Predictive Maintenance & Analytics	Anomaly detection and remaining-useful-life models; model training, deployment, monitoring and retraining.
6. Energy & Resource Optimisation	Tracks consumption per machine/plant; automatic load scheduling and wastage alerts.
7. Dashboard, Reporting & Alerts	Real-time web and mobile dashboards, historical trends, automated reports (PDF/Excel), SMS / e-mail / push alerts.
8. Integration & API Gateway	REST/GraphQL APIs and connectors to ERP, MES, CMMS, e-mail and third-party services; rate limiting and versioning.
9. User, Role & Security Management	Authentication (OAuth2 / JWT, MFA), role-based access control, audit logs, encryption key management.
10. Platform Operations (DevOps)	CI/CD pipeline, infrastructure as code, centralised logging, monitoring, auto-scaling, backup and disaster recovery.

4.3 Requirements

4.3.1 Functional requirements

FR1: The system shall collect sensor and machine data automatically at configurable intervals from all registered devices.

FR2: The system shall allow administrators to register, configure, group and deactivate devices and gateways remotely.

FR3: The system shall allow authorised users to define rules, thresholds, schedules and workflows without writing code.

FR4: The system shall generate real-time alerts via dashboard, e-mail, SMS and mobile push when a rule is violated.

FR5: The system shall automatically create, assign and track maintenance work orders and send them to the CMMS.

FR6: The system shall predict equipment failure and display a health score with recommended actions.

FR7: The system shall allow authorised users to send commands (start, stop, set-point) to devices with confirmation and logging.

FR8: The system shall provide live dashboards, historical trend analysis and downloadable reports per plant, line and machine.

FR9: The system shall support role-based access (Admin, Manager, Engineer, Operator, Viewer) with multi-factor authentication.

FR10: The system shall integrate with ERP/MES/CMMS through documented APIs and keep records synchronised.

FR11: The system shall maintain a tamper-evident audit log of all user actions and automated decisions.

FR12: The system shall continue critical local rules at the edge when connectivity to the cloud is lost and synchronise on reconnection.

4.3.2 Non-functional requirements

Quality Attribute	Requirement
Performance	Alert generation within 2 seconds of the event; dashboard pages load in under 3 seconds; ingestion capacity of at least 50,000 events per second with elastic growth.
Scalability	Horizontal scaling of services; onboarding a new plant or 10,000 additional devices without architectural change.

Quality Attribute	Requirement
Availability	Minimum 99.9% uptime; deployment across multiple availability zones; Recovery Time Objective (RTO) of 1 hour and Recovery Point Objective (RPO) of 15 minutes.
Reliability	No data loss during network outage (edge buffering); automatic retry and failover; self-healing containers.
Security	TLS 1.2+ in transit, AES-256 at rest, least-privilege access, MFA, periodic vulnerability scans, compliance with ISO 27001 / IEC 62443 practices.
Usability	Simple, responsive interface usable on mobile; maximum three clicks to reach any critical view; multilingual support; short training time.
Maintainability	Modular microservices with documented APIs; code coverage above 80%; automated builds and zero-downtime releases.
Interoperability	Standard protocols (MQTT, OPC-UA, REST) and open data formats (JSON, CSV) to avoid vendor lock-in.
Portability	Containerised services that can run on any major cloud or on-premise Kubernetes.
Cost efficiency	Auto-scaling, storage tiering and budget alerts to keep cloud spending within approved limits.

4.4 Cloud Architecture Decisions

Decision	Choice and Reason
Deployment model	Hybrid (edge + public cloud): control loops stay local for low latency and safety; analytics and orchestration use cloud elasticity.
Service model	Mix of IaaS (Kubernetes), PaaS (managed databases, streaming, IoT hub) and SaaS (monitoring) to reduce operational effort.
Architecture style	Microservices with event-driven communication; each service owns its data and can be deployed independently.
Compute	Containers on Kubernetes with auto-scaling; serverless functions for short event-triggered tasks.
Data	Polyglot persistence: time-series DB, relational DB and object storage chosen for the type of data.
Resilience	Multi-AZ deployment, load balancers, health checks, circuit breakers, cross-region backup.
Security model	Zero-trust: device certificates, API gateway authentication, network segmentation, encrypted secrets.
Operations	Infrastructure as Code, GitOps, centralised logging and metrics, cost dashboards.

4.5 Application of Software Engineering Principles

Principle	How It Is Incorporated in SCAP	Benefit
Modularity	System divided into independent microservices (ingestion, rules, ML, alerts, users) with well-defined APIs; high cohesion and low coupling; layered design.	Teams work in parallel; a change in one module does not break others; easy reuse.
Scalability	Stateless services, horizontal auto-scaling, partitioned event streams, managed elastic storage, edge pre-processing to reduce load.	Handles growth from five plants to fifty without redesign.

Principle	How It Is Incorporated in SCAP	Benefit
Maintainability	Clean architecture, coding standards, code reviews, automated unit/integration tests, API versioning, infrastructure as code, complete documentation and logs.	Faster bug fixes and enhancements; lower long-term cost.
Reliability	Redundancy across availability zones, health checks, retries with back-off, circuit breakers, edge buffering, backups and tested disaster-recovery plan.	Continuous operation even during failures.
Usability	Role-based dashboards, simple low-code rule builder, consistent UI, mobile-first design, user testing in each sprint.	Low training effort and higher adoption.
Security	Security by design: MFA, RBAC, TLS, encryption at rest, secrets vault, device identity, input validation, audit logs, regular penetration testing.	Protects data, devices and the business; supports compliance.
Abstraction & Information Hiding	Protocol adapters hide device differences; services expose only APIs, not internal data stores.	New device types or databases can be added with minimal change.
Separation of Concerns	Presentation, business logic, data and infrastructure are separated into layers.	Simpler design and testing.
Testability & Quality	Test pyramid (unit, integration, system), simulated device test-bench, CI quality gates.	Defects are found early.

5. Justification of the Chosen Methodology / Framework

5.1 Selected Model: Agile (Scrum) with DevOps (CI/CD)

For SCAP, the selected approach is **Agile using the Scrum framework, combined with DevOps practices** (continuous integration, continuous delivery, infrastructure as code and continuous monitoring). Work is organised in **two-week sprints**, with a Product Owner (plant operations head), a Scrum Master and a cross-functional development team of 6–8 members.

Suggested Scrum set-up

- **Product Backlog:** prioritised list of features (user stories) such as "As a maintenance engineer, I want an alert when vibration exceeds the limit so that I can act before breakdown."
- **Sprint cycle:** sprint planning → daily 15-minute stand-up → development and testing → sprint review with plant users → sprint retrospective.
- **Release plan (example):** Sprint 1–2: device onboarding, secure ingestion; Sprint 3–4: dashboard and alerts; Sprint 5–6: rules/workflow engine and ERP integration; Sprint 7–8: predictive maintenance and energy module; Sprint 9: security hardening, load testing and pilot at one plant; then plant-by-plant rollout.
- **Definition of Done:** code reviewed, automated tests passed, security scan clean, documentation updated, deployed to staging.

5.2 Why This Methodology Is Suitable

- **Requirements will evolve.** Operators and managers will discover new needs after seeing working dashboards and alerts. Scrum welcomes change at every sprint, while Waterfall freezes requirements too early.
- **Early and continuous value.** Each sprint delivers a working increment (for example, live monitoring for one line), so benefits begin in weeks rather than after a long single delivery.

- **Risk reduction.** Integration with legacy devices and cloud services is uncertain. Short iterations, a pilot plant and frequent reviews expose technical risks early, and a Spiral-style risk review can be carried out during backlog grooming.
- **Close stakeholder collaboration.** Sprint reviews give plant managers, operators and security officers a regular voice, improving usability and adoption.
- **Cloud and automation fit DevOps naturally.** Infrastructure as code, automated testing and CI/CD pipelines allow frequent, safe, zero-downtime releases, which suits a platform that must run 24x7.
- **Team and scale.** A modular microservice product can be split among small teams, each owning a service, which matches the Scrum team structure.

5.3 Comparison with Other Models

Model	Limitation for this Project	Verdict
Waterfall	Rigid; changes are costly; working software appears only at the end; poor fit for unclear device and integration requirements.	Not suitable
Spiral	Strong risk analysis but heavy documentation and management overhead; costly for a mid-sized team.	Partly useful – its risk practices are adopted inside Scrum
V-Model	Good testing discipline but sequential and inflexible.	Not suitable
Agile (Scrum) + DevOps	Needs disciplined teams and active user involvement.	Best fit – adaptive, iterative, supports continuous delivery

5.4 How the Methodology Supports Development and Maintenance

- **Development:** prioritised backlog, timeboxed sprints, transparent progress through burndown charts, and continuous integration keep the schedule and quality under control.
- **Quality:** test automation, code reviews and security scans in the pipeline (DevSecOps) catch defects early.
- **Deployment:** blue-green or canary releases with automatic rollback reduce release risk.
- **Maintenance:** the same team supports the product ("you build it, you run it"); monitoring data and user feedback flow directly back into the backlog, so maintenance becomes continuous improvement rather than a separate phase.
- **Knowledge sharing:** retrospectives and living documentation reduce dependency on individuals.

6. Expected Outcomes and Limitations

6.1 Expected Benefits

Benefit	Expected Result (targets to be validated in the pilot)
Reduced unplanned downtime	Predictive alerts allow repairs before failure; target 25–35% reduction in breakdown hours.
Higher productivity	Automation removes manual logging and approvals; target 30–40% less administrative effort.
Real-time visibility	Management sees all plants in a single dashboard within seconds instead of days.
Cost savings	Energy optimisation (target 8–12%), reduced emergency repairs, and no capital spend on new servers.
Scalability and agility	New plants and devices can be onboarded in days instead of months.

Benefit	Expected Result (targets to be validated in the pilot)
Better security & compliance	Central access control, encryption and audit logs lower security incidents and ease audits.
Data-driven decisions	Analytics reveal trends, bottlenecks and waste that were invisible earlier.
Improved quality and safety	Faster detection of abnormal conditions reduces defects and hazards.

6.2 How the Solution Addresses the Identified Challenges

Challenge	How SCAP Addresses It
Data silos	Central cloud data platform and integration APIs combine data from all plants and systems.
Manual effort and errors	Automatic data capture and the workflow engine replace manual logs and e-mail approvals.
Unplanned downtime	Predictive maintenance models and condition-based alerts.
Limited scalability	Containers, auto-scaling and managed cloud services.
Legacy / heterogeneous devices	Protocol adapters and edge gateways standardise data from all devices.
Security risk	Zero-trust design, device certificates, MFA, encryption and continuous monitoring.
Unstable connectivity	Edge buffering and local rules continue operation offline and sync later.
Cost uncertainty	Pay-as-you-go model with budgets, tagging and cost dashboards.
Change resistance	Simple interface, user involvement in sprint reviews, and training plan.

6.3 Potential Risks and Implementation Challenges

Risk / Challenge	Impact	Mitigation
Integration with legacy equipment	Delays, incomplete data	Early proof-of-concept, protocol gateways, pilot at one plant.
Cybersecurity attack on devices or cloud	Data breach, operational disruption	Zero-trust, network segmentation, patching, pen-testing, incident-response plan.
Cloud cost overruns	Budget pressure	Budget alerts, right-sizing, storage tiering, reserved capacity.
Vendor lock-in	Difficult migration later	Open standards, containers, abstraction layers.
Network outage	Loss of remote visibility	Edge buffering and local rule execution; redundant connectivity.
Poor data quality / model drift	Wrong predictions, false alarms	Data validation, monitoring, scheduled retraining.
User resistance, skill gap	Low adoption	Training, champions at each plant, phased rollout.
Scope creep	Schedule and cost overrun	Prioritised backlog, product-owner control, sprint reviews.
Regulatory / data-residency rules	Compliance issues	Choose in-country cloud regions; privacy-by-design; regular audits.

6.4 Limitations of the Proposed Solution

- Dependence on internet connectivity for cloud analytics and remote control; only edge-level functions work during an outage.
- Round-trip latency to the cloud makes it unsuitable for hard real-time, safety-critical control, which must stay on local controllers.
- Predictive models need sufficient historical data; accuracy will be low in the first months.
- Ongoing operating cost (cloud subscription, monitoring, skilled staff) replaces the earlier one-time capital cost.
- Initial effort to retrofit old machines with sensors and gateways can be significant.
- The solution assumes management commitment and staff willingness to change established practices.

6.5 Future Enhancements

- **Digital twin:** virtual models of machines and lines for simulation and "what-if" analysis.
- **5G and edge AI:** run ML models directly on gateways for faster local decisions.
- **Generative-AI assistant:** natural-language queries such as "why did Line 3 stop yesterday?" and automatic root-cause summaries.
- **Computer-vision quality inspection** integrated into the automation workflows.
- **Blockchain-based traceability** for supply chain and compliance records.
- **Multi-cloud and disaster-recovery automation** for greater resilience and cost optimisation.
- **Sustainability dashboard:** carbon-footprint tracking and automatic optimisation to meet ESG goals.
- **Marketplace of reusable automation templates** to onboard new plants even faster.

7. Conclusion

Traditional, siloed and manual operations cannot meet today's demand for speed, reliability and insight. A smart automation platform built on a layered, cloud-native architecture with an edge component removes manual effort, predicts failures, unifies data and scales on demand, while preserving local safety and offline operation. By applying core software engineering principles (modularity, scalability, maintainability, reliability, usability and security) and delivering the system through Agile Scrum supported by DevOps, the organisation can build SCAP incrementally, reduce risk, show value from the first sprints and keep improving the product after release. Despite limitations such as connectivity dependence and ongoing cloud cost, the benefits clearly outweigh the risks, making the proposed solution technically feasible, economically sensible and strategically future-ready.

References

1. Sommerville, I. (2016). Software Engineering (10th ed.). Pearson.
2. Pressman, R. S., & Maxim, B. R. (2019). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill.
3. Schwaber, K., & Sutherland, J. The Scrum Guide. scrumguides.org.
4. Amazon Web Services. AWS Well-Architected Framework. aws.amazon.com/architecture/well-architected.
5. Microsoft. Azure Architecture Center: Cloud Design Patterns. learn.microsoft.com/azure/architecture.
6. Kim, G., Humble, J., Debois, P., & Willis, J. (2016). The DevOps Handbook. IT Revolution Press.
7. IEC 62443: Industrial communication networks – Network and system security.